

**MBA IN SOFTWARE
ENGINEERING**

Domain Driven Design (DDD) II

Prof. Guilherme Lima

MBA **USP**
ESALQ

The trustworthiness, originality,
and legality of the educational content presented is responsibility of the professor.

The total or partial reproduction of this material without authorization is prohibited.

Law No.
9610/98



Class 2

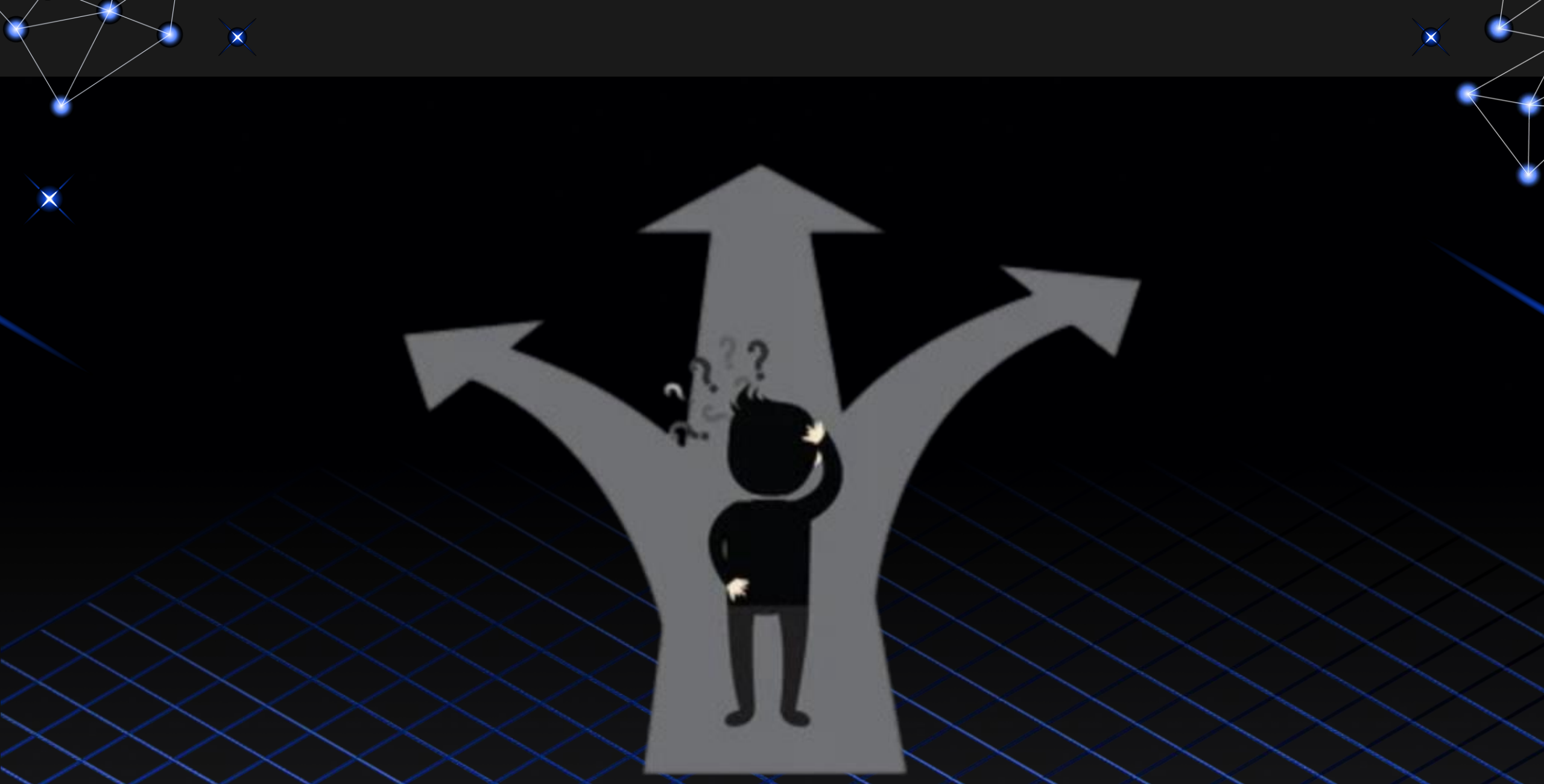
Domain-Driven Designer

Professor *Guilherme Lima*





DDD is not
exclusively about
technology



Value Objects

DOMAIN-DRIVEN DESIGN / TACTICAL PATTERNS

Represent everything with primitive types

Primitive types only represent values, without behavior or validation, while classes allow:

- encapsulate business rules
- ensure data consistency
- model domain entities as objects



Business Rules:
unique,
spam filter,
delivery status,
audit log

email
(primitive type)

Email
(class)

@ ✓
Validations:
regex, format,
existence
check

```
Python  
for i  
pyt  
}  
jav  
con  
print
```

Invalid email and ZIP codes enter without resistance

Without **Value Objects**, validation is a loose function that someone needs to remember to call.

Email:

jondoe.email.com ✓

ZIP CODE:

1234 ✓

INVALID DATA ACCEPTED

System **trust** is lost when
we allow primitive types to
represent business rules.



Let's analyze project-1 without DDD

The solution: each type becomes a Value Object

Instead of generic types, each concept gets its own class. The type now carries the **business rules** within itself.



Email



ZIP CODE



Address



PROJECT 1

DOMAIN MODEL

DOMAIN-DRIVEN DESIGN / TACTICAL PATTERNS

THE 3 RULES OF THE LIBRARY

01

Maximum Capacity

The collection cannot exceed the limit of 5 simultaneous books.

02

Availability

A book marked as unavailable cannot be borrowed.

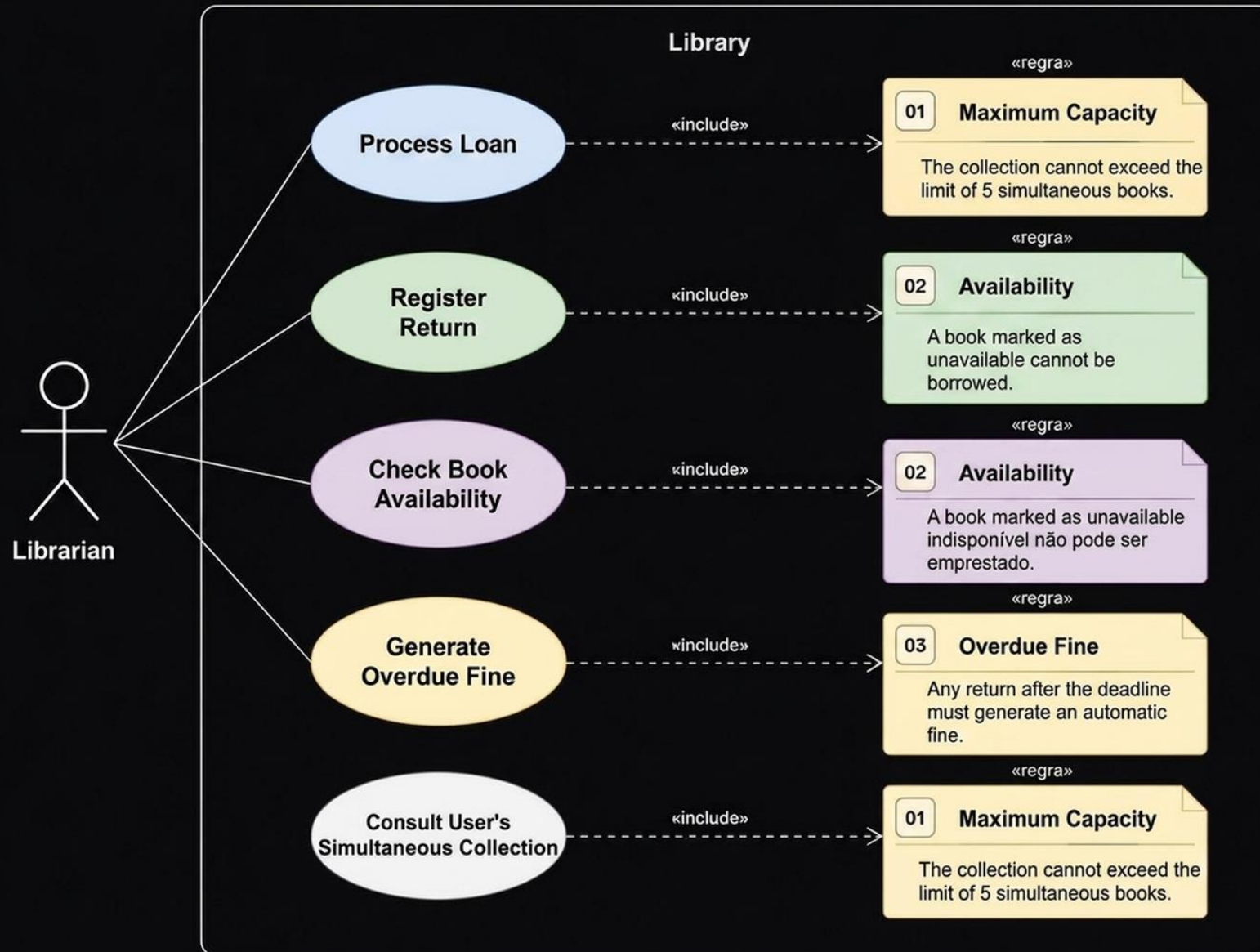
03

Overdue Fine

Any return after the deadline must generate an automatic fine.

Invariant: A business rule that must be guaranteed in any system state.

USE CASE DIAGRAM - LIBRARY RULES (INVARIANTS)



Let's analyze project-2 without DDD

Problem 1 — THE COLLECTION REACHES 6 BOOKS

6

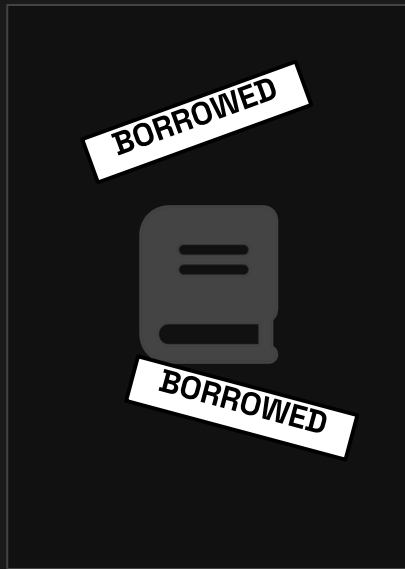
LIMIT: 5

Since the list of books is **public**, any part of the system can add items directly.

```
library.books.append(new_book);
```

The business rule exists on paper, but the code does not offer resistance to state corruption.

Problem 2 — THE SAME BOOK BORROWED TWICE



INVALID STATE

Data inconsistency in the database

Problem 3 — THE FINE THAT NO ONE GUARANTEES TO APPLY

**fine = loose bill, easy to
forget**

THE SOLUTION: LIBRARY AS AGGREGATE ROOT



AGGREGATE ROOT

THE CONSISTENCY BOUNDARY: FROM THE OUTSIDE, IT'S READ-ONLY



READING

The state is **public and freely accessible** by any part of the system.



WRITING

Changes are **restricted** to the public methods of the Aggregate Root.

EACH COMMAND PROTECTS AN INVARIANT



`add_book()`

Checks the current size of the collection and refuses the operation if the limit of 5 is reached.



`borrow()`

Validates the book's availability status before allowing the state transition.



`return()`

Calculates and automatically applies the fine based on the return date and deadline.



PROJECT 3

Anti-Corruption Layer

DOMAIN-DRIVEN DESIGN / TACTICAL PATTERNS

LEGACY
SYSTEM —
DO NOT CHANGE!

Almost every system needs to communicate with something we **don't control**:

- a bank's API
- an old library
- a third-party system



Let's analyze project-3

Thank you!

Prof. Guilherme Lima | <https://www.linkedin.com/in/guilherme-lima-developer>

MBA **USP**
ESALQ